山东大学 计算机科学与技术 学院

云计算技术 课程实验报告

学号：202200130077	姓名：薛润泽	班级：智能 22 级
实验题目：实现 RSA 加解密及破解		
实验学时：2	实验日期： 2025.5.7	

实验目的：实现 RSA 加解密实验，认识 RSA 加解密过程。

具体内容：

1）编程实现 RSA 加密算法，包括密钥生成、加密和解密过程；

2）基于因式分解进行破解，注意为了保证破解的可行性，需要选取合适的秘钥长度，请选择 5 种不同的秘钥长度，并给出对应的破解时间。

硬件环境：

计算机一台

软件环境：

Linux 或 Windows

实验步骤与内容：

1、RSA 原理？

RSA 是一种非对称加密算法。核心在于加密密钥和解密私钥的获取。

其中，假设明文为 m，则加密过程为 $m^e \equiv c \ (\% \ n)$

解密过程为 $c^d \equiv m \ (\% \ n)$

将加密解密写在一起可以得到 $m^{ed} \equiv m \ (\% \ n)$

2、实验具体过程

（1）生成两个大质数 p, q。则 n = p * q

（2）使用 p, q 生成密钥对。

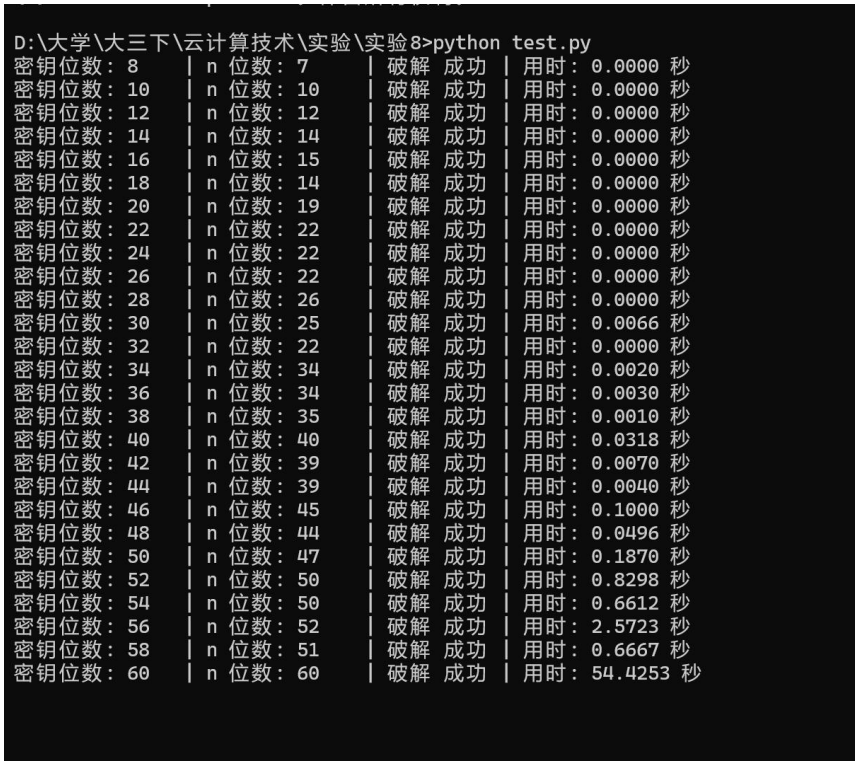
首先公钥需要随机寻找一个与 n 互质的质数 a。因为 p，q 都是质数，所以 n 的欧拉函数为 (p-1)*(q-1)，理论上私钥计算为 $b = a^{(p-1)*(q-1)}$，不过这个数往往比较大，非常难计算。所以使用扩展欧几里得算法进行私钥计算。

（3）因式分解破解私钥

因为公钥给出了 a, n，且 n 的公因数为 p,q 而私钥计算理论上为 $b = a^{(p-1)*(q-1)}$,所以只要能将 n 因式分解回 p，q 即可完成破解。

结论分析与体会：

尝试使用不同位数的秘钥进行加密和破解，统计时间数据如下：



代码附录：

```
import random
import time
from sympy import isprime, nextprime, gcd
from math import gcd as math_gcd

def generate_prime(bits):
    p = random.getrandbits(bits)
    while not isprime(p):
        p = nextprime(p)
    return p

def generate_keypair(bits):
    p = generate_prime(bits)
    q = generate_prime(bits)
    while p == q:
        q = generate_prime(bits)
    n = p * q
    phi = (p - 1) * (q - 1)
    e = 65537
    while math_gcd(e, phi) != 1:
```

```
            e = nextprime(e)
        d = pow(e, -1, phi)
        return ((e, n), (d, n))

def encrypt(msg, pubkey):
        e, n = pubkey
        return pow(msg, e, n)
def decrypt(cipher, privkey):
        d, n = privkey
        return pow(cipher, d, n)

def factor_rsa(n):
        for i in range(2, int(n**0.5) + 1):
                if n % i == 0:
                        return i, n // i
        return None, None

def crack_rsa(n, e):
        p, q = factor_rsa(n)
        if p is None:
                return None
        phi = (p - 1) * (q - 1)
        d = pow(e, -1, phi)
        return d

def test_crack(bits):
        pub, priv = generate_keypair(bits)
        e, n = pub
        d_real, _ = priv

        start = time.time()
        d_cracked = crack_rsa(n, e)
        end = time.time()

        cracked = d_cracked == d_real
        duration = end - start

        print(f"密钥位数: {bits*2:<4} | n 位数: {n.bit_length():<5} | 破解 {'成功' if cracked else '失败'} | 用时: {duration:.4f} 秒")
        return duration
key_sizes = []
for i in range(4, 128):
        key_sizes.append(i)
times = []
```

```
for bits in key_sizes:
    t = test_crack(bits)
    times.append(t)
```